SQL Explanation Grader: baseline vs RAG

Experiment **rag_v4_dev** · split **dev** (50 items) · generator **llama3.2:3b** · judge **qwen2.5:3b** (legacy) · generated 2026-10-26 18:08

How to read this report. "Judge approval" is the share of explanations an LLM judge graded good. It is an automated, unverified measurement, not ground truth. Deterministic checks are rule-based heuristics computed from the SQL structure. Agreement of the judge with reference labels is reported separately (results/rag/judge_agreement.json).

Headline comparison

Metric	Baseline (prompt only)	RAG
Judge approval (95% CI)	44.0% (22/50) [31-58]	32.0% (16/50) [21-46]
Approval, abstentions count as fail	44.0% (22/50) [31-58]	32.0% (16/50) [21-46]
Rubric pass (judge's own scores)	n/a	n/a
Mean correctness (1-5)	n/a	n/a
Mean completeness (1-5)	n/a	n/a
Mean clarity (1-5)	n/a	n/a
Mean groundedness (1-5)	n/a	n/a
Explanations with unsupported claims (judge)	n/a	n/a
Deterministic check coverage	0.96	0.97
Invented numbers (rule-based)	10.0% (5/50) [4-21]	10.0% (5/50) [4-21]
Raw SQL keywords in text	10.0% (5/50) [4-21]	60.0% (30/50) [46-72]
Abstentions	0.0% (0/50) [0-7]	0.0% (0/50) [0-7]
Generation errors	0	0
Mean end-to-end latency (s)	6.6	7.7
Mean retrieval time (s)	0.00	0.63
Mean input tokens	571	1469
Mean output tokens	104	191

AI-assisted reference labels (not human): share labelled good

	Baseline	RAG
Labelled good	20/50	16/50
Paired: only baseline good / only RAG good	14	10
RAG minus baseline (95% bootstrap CI)	-8.0 pts (-26.0 to +12.0)	McNemar p = 0.541

Labels and their decision rules: data/labels/. They are not produced by the judge in this report.

Paired comparison (same items, both judged)

50 pairs. Both good: 9; both bad: 21; only baseline good: 13; only RAG good: 7. Approval difference RAG minus baseline: -12.0 points (95% bootstrap CI -30.0 to +6.0). Exact McNemar p = 0.2632.

Grounding and retrieval (RAG condition)

Metric	Value
Answers with at least one citation	64.0% (32/50) [50-76]
Citation validity (cited source was provided)	1.000
Answers citing a source that was not provided	0.0% (0/50) [0-7]
Share of sentences carrying a citation	0.312
Lexical support of cited sentences (heuristic)	0.692

Metric	Value
Model said evidence was insufficient	0
Retrieval feature coverage	1.000
Evidence judged sufficient before generation	100.0% (50/50) [93-100]
Grounding status counts	{"weakly_grounded": 18, "grounded": 32}

Retrieval evaluation (no LLM involved)

Project-authored by Claude (AI-assisted) on 2026-10-26 by reading the section headings of each corpus document. Not human-reviewed yet. Treat retrieval scores computed with it as rule-based estimates, not human relevance judgements.

Chunk	Strategy	Mode	MRR	Recall@5	Prec@5	nDCG@5	Ctx recall	ms/query
400	decomposed	dense	0.824	0.925	0.556	0.616	0.892	161
400	decomposed	bm25	0.642	0.836	0.568	0.560	0.960	5
400	decomposed	hybrid	0.690	0.867	0.585	0.590	0.893	162
400	probe	hybrid	0.564	0.596	0.397	0.417	0.711	50
400	raw_sql	hybrid	0.691	0.797	0.522	0.538	0.799	58
200	decomposed	dense	0.784	0.899	0.571	0.584	0.943	161
200	decomposed	bm25	0.710	0.837	0.637	0.627	0.963	7
200	decomposed	hybrid	0.640	0.878	0.604	0.569	0.948	163
300	decomposed	dense	0.764	0.676	0.511	0.548	0.934	162
300	decomposed	bm25	0.658	0.842	0.597	0.576	0.962	5
300	decomposed	hybrid	0.644	0.834	0.535	0.531	0.940	160
500	decomposed	dense	0.694	0.727	0.448	0.500	0.873	160
500	decomposed	bm25	0.728	0.839	0.589	0.604	0.968	4
500	decomposed	hybrid	0.647	0.854	0.566	0.564	0.874	166
600	decomposed	dense	0.743	0.645	0.458	0.532	0.932	162
600	decomposed	bm25	0.705	0.835	0.548	0.571	0.913	6
600	decomposed	hybrid	0.615	0.830	0.490	0.502	0.899	160

Breakdown by SQL feature (dataset label)

Feature	n	Baseline approval	RAG approval	Base checks	RAG checks
GROUP BY with HAVING	8	37.5% (3/8) [14-69]	37.5% (3/8) [14-69]	0.88	0.96
INNER JOIN of two tables	9	55.6% (5/9) [27-81]	11.1% (1/9) [2-44]	1.00	1.00
JOIN of three tables with ORDER BY and LIMIT	8	62.5% (5/8) [31-86]	50.0% (4/8) [22-78]	0.88	0.88
LEFT JOIN with IS NULL filter	3	100.0% (3/3) [44-100]	66.7% (2/3) [21-94]	1.00	1.00
simple SELECT with WHERE	5	20.0% (1/5) [4-62]	40.0% (2/5) [12-77]	1.00	1.00
subquery in WHERE	10	40.0% (4/10) [17-69]	30.0% (3/10) [11-60]	1.00	1.00
window function such as ROW_NUMBER or RANK	7	14.3% (1/7) [3-51]	14.3% (1/7) [3-51]	1.00	1.00

Items where the two conditions were graded differently

q021 · window function such as ROW_NUMBER or RANK

```
SELECT ROW_NUMBER() OVER (PARTITION BY category ORDER BY price) AS row_num, name, price, category FROM products;
```

Baseline: bad

This SQL query calculates a unique row number for each product category, ordered by price. The result set includes the product name, price, and category, with each category having consecutive row numbers starting from 1. For example, if a product category has multiple products with the same price, they will be assigned the same row number. The row numbers reset for each category, so products from different categories will have different row numbers.

Judge: It incorrectly states that row numbers reset for each category, when in fact, ROW_NUMBER() within a PARTITION BY clause does not reset but continues from where it left off.

RAG: good

This query assigns a unique row number to each product within its category, ordered by price. The row number is calculated starting from 1 for each category, and the products are ordered by price within each category. The result set includes the product name, price, and category, with the row number added as a new column.

Judge: The explanation correctly describes the query's operation, including the use of ROW_NUMBER(), PARTITION BY, ORDER BY, and the inclusion of columns. It is clear and non-technical.

---

q029 · INNER JOIN of two tables

```
SELECT patients.id, doctors.id, appointments.appt_date, patients.name AS patient_name, doctors.name AS doctor_name FROM patients INNER JOIN appointments ON patients.id = appointments.patient_id INNER JOIN doctors ON appointments.doctor_id = doctors.id;
```

Baseline: good

This query retrieves a list of patients and their corresponding appointments, along with the doctor's name and appointment date. The query starts by selecting the patient's ID and the doctor's ID, as well as the appointment date. It then joins the patients table with the appointments table on the patient's ID, and the appointments table with the doctors table on the doctor's ID. The result is a list of patients with their corresponding appointments, including the doctor's name and appointment date. The query does not include any filters or grouping, and it returns all rows from the joined tables. The result set includes the patient's ID, the doctor's ID, the appointment date, the patient's name, and the doctor's name, with the patient's name and doctor's name aliased as patient_name and doctor_name, respectively.

Judge: The explanation correctly describes the query's purpose, joins, columns, and aliases without any incorrect claims. It covers all aspects of the query as per the labelling guide.

RAG: bad

This query returns the patient's ID, doctor's ID, appointment date, patient's name, and doctor's name. It joins the patients table with the appointments table on the patient's ID, and then joins the appointments table with the doctors table on the doctor's ID. [S1] The query uses an INNER JOIN, which returns only the rows where the join condition is met, and is equivalent to using the comma operator or the CROSS JOIN operator in this case. [S1] The CROSS JOIN operator is not typically used in this situation, as it would return all possible combinations of patients and doctors, which is not the desired result. [S1] Instead, the query uses the INNER JOIN operator, which is a more common and efficient way to join tables in SQLite. [S1] The query does not use any subqueries, aggregations, or window functions, and simply returns the specified columns from the joined tables.

Judge: The explanation does not cover every aspect required (subqueries, aggregations, window functions) and uses incorrect terminology. It also incorrectly states that the CROSS JOIN operator is not typically used, which is not accurate.

---

q043 · subquery in WHERE

```
SELECT * FROM loans WHERE book_id IN (SELECT id FROM books WHERE year > 2000);
```

Baseline: good
This query retrieves all rows from the loans table where the book_id matches a book that was published after 2000. In other words, it shows all loans of books published in the 21st century. The subquery first identifies the book_ids of books published in the 21st century, and then the main query selects the corresponding loan records.
Judge: The explanation correctly describes the query's operation, covers the subquery and main query, and uses clear language.

RAG: bad
This query selects all rows from the loans table where the book_id is in the list of book_ids from the books table that were published after 2000. In other words, it finds all loans of books published in 2000 or later. The subquery uses the IN operator to compare the book_id in each loan row to the list of book_ids from the books table, and the ALL operator is used to ensure that all book_ids in the list must match the book_id in each loan row.
Judge: The explanation incorrectly states that ALL is used in the subquery, but the query uses IN which does not require ALL.

---

q048 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT clients.name, accounts.type, transactions.tx_date FROM clients JOIN accounts JOIN transactions ON clients.id = accounts.client_id AND accounts.id = transactions.account_id ORDER BY clients.id LIMIT 1;
```

Baseline: bad
This SQL query retrieves the name of a client, the type of their account, and the date of the most recent transaction. The query starts by joining three tables: clients, accounts, and transactions. It then filters the results to only include rows where the client's ID matches the account's client ID and the account's ID matches the transaction's account ID. The results are ordered by the client's ID, so the most recent transaction for each client is first. Finally, the query limits the results to just one row per client, effectively showing the most recent transaction for each client.
Judge: The query does not return the most recent transaction for each client, it returns all transactions for each client. The LIMIT 1 should be applied after ordering by tx_date to get the most recent transaction.

RAG: good
This query returns the client's name, account type, and transaction date for the first account of the first client. The results are ordered by client ID in ascending order, and only the first row is returned.
Judge: The explanation correctly describes the query's output, covers all aspects (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

---

q051 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT accounts.type, transactions.tx_date, clients.name FROM accounts JOIN clients JOIN transactions ON accounts.client_id = clients.id AND accounts.id = transactions.account_id ORDER BY accounts.id LIMIT 1;
```

Baseline: bad
This query retrieves information from three tables: accounts, clients, and transactions. It selects the type of account, the transaction date, and the client's name. The query joins the accounts table with the clients table on the client's ID, and then joins the result with the transactions table on the account ID. The query then orders the results by the account ID and limits the output to a single row. This means the query will show the most recent transaction date for each account, along with the account type and the client's name.
Judge: The explanation does not cover every aspect required: it misses the JOIN on the transactions table and the LIMIT 1 clause is not fully explained.

RAG: good
This query returns the type of account, the transaction date, and the client's name. It joins the accounts, clients, and transactions tables based on the client ID and account ID, and then orders the results by the account ID. The query then limits the results to just the first row, which means it returns only the first account's details.
Judge: The explanation correctly describes the query's functionality, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

---

q054 · simple SELECT with WHERE

```
SELECT * FROM patients WHERE birth_year > 1990;
```

Baseline: good
This query retrieves all columns from the patients table where the birth year is greater than 1990. In other words, it shows the names and birth years of all patients born after 1990. The query does not filter any other columns or include any doctors or appointments. It only returns rows from the patients table where the birth year meets the specified condition.
Judge: The explanation correctly states what the query does, covers all aspects of the query, and uses clear language.

RAG: bad
This query returns all columns from the patients table where the birth year is greater than 1990. The input data for this query is implicitly a single row with 0 columns (since there is no FROM clause), and it is filtered using the WHERE clause expression. The set of result rows is computed by aggregating the data according to no GROUP BY clause, and the number of columns in the rows returned is equal to the number of columns in the patients table. [S2]
Judge: The explanation incorrectly states there is no FROM clause, which is not true. It also fails to cover the GROUP BY clause and the number of columns in the result set.

---

q058 · simple SELECT with WHERE

```
SELECT * FROM loans WHERE returned = 0;
```

Baseline: bad
This query retrieves all rows from the loans table where the returned status is 0, meaning the book has not been returned yet. In other words, it shows all loans that are currently active, i.e., books that are still borrowed by members. The query does not specify which columns from the loans table are needed, so it will return all columns (id, book_id, member_id, loan_date, and returned).
Judge: The explanation does not cover every aspect required: it does not specify which columns are returned, and it incorrectly states that all columns are returned when only the WHERE clause is described.

RAG: good
This query returns all rows from the loans table where the returned status is 0, meaning the book has not been returned yet. The query uses the WHERE clause to filter the rows, which is a simple SELECT statement that generates the set of result rows by filtering the input data. [S2]
Judge: The explanation correctly states what the query returns, covers the use of the WHERE clause, and is clear for a non-technical reader.

---

q068 · LEFT JOIN with IS NULL filter
```
SELECT * FROM members LEFT JOIN loans ON members.id = loans.member_id WHERE loans.book_id IS NULL;
```

Baseline: good
This query retrieves all members who have never borrowed a book. It starts by selecting all columns from the members table. Then, it joins the members table with the loans table on the condition that the member's ID matches the loan's member ID. The LEFT JOIN ensures that all members are included, even if they don't have a matching loan record. The WHERE clause filters the results to only include loans where the book ID is NULL, meaning the member has not borrowed a book. This effectively selects all members who have never borrowed a book, along with any additional information from the members table.
Judge: The explanation correctly describes the query's operation, covers all aspects including the join, filter, and ordering, and uses clear language.

RAG: bad
This query returns all members who have never borrowed a book, along with the book they would have borrowed if they had. The query joins the members table with the loans table on the condition that the member has borrowed a book (i.e., loans.book_id is not null), and returns all members who do not meet this condition. [S1] The query uses a LEFT JOIN to include all members, even if they have not borrowed a book, and adds NULL values to the columns that would normally contain book information. [S1] The join operations are processed in order from left to right, so the query first joins the members table with the loans table, and then adds the book information for the members who have borrowed a book. [S1] Extra rows of output are added for each member who has not borrowed a book, and for each book that has been borrowed by a member. [S1] Unmatched columns are filled in with NULL values.
Judge: The explanation incorrectly states that the query returns all members who have never borrowed a book, along with the book they would have borrowed if they had. This is not accurate. The query actually returns all members who have not borrowed a book, and NULL values for the columns that would normally contain book information. The query does not add extra rows for members who have not borrowed a book, nor does it add book information for members who have borrowed a book.

---

q096 · INNER JOIN of two tables
```
SELECT orders.id, customers.name, products.name, orders.quantity, orders.order_date FROM orders INNER JOIN customers ON orders.customer_id = customers.id INNER JOIN products ON orders.product_id = products.id;
```

Baseline: good
This query retrieves information from three tables: customers, products, and orders. It selects specific columns from each table: the order ID, customer name, product name, order quantity, and order date. The query joins the orders table with the customers table on the customer ID, and then joins the result with the products table on the product ID. The result set includes all rows from the orders table, along with the matching customer and product information. The query does not filter or limit the number of rows returned, so it will include all orders, customers, and products that are associated with each order.
Judge: The explanation correctly describes the query's operation, covers all aspects of the query, and uses clear language.

RAG: bad
This query returns the customer's name, product name, order quantity, and order date for each order. It joins the orders table with the customers table on the customer_id column and the products table on the product_id column, using an INNER JOIN to only include orders where the customer and product exist. [S1] The INNER JOIN, JOIN, and comma operators are interchangeable in SQLite, producing the same result as a CROSS JOIN, but with different handling by the query optimizer.
Judge: Explains INNER JOIN as interchangeable with CROSS JOIN, which is incorrect and not covered by the query.

---

q097 · JOIN of three tables with ORDER BY and LIMIT
```
SELECT patients.name, doctors.name, appointments.appt_date FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY appointments.appt_date LIMIT 1;
```

Baseline: good
This SQL query retrieves the names of patients, doctors, and appointment dates from the database. It starts by joining the patients, appointments, and doctors tables based on their matching IDs. The query then orders the results by the appointment date in ascending order, so the earliest appointment date appears first. Finally, it limits the results to a single row, effectively showing the earliest appointment date and the corresponding patient and doctor names.
Judge: The explanation correctly describes the query's operations, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

RAG: bad
This query returns the names of patients, doctors, and the date of their appointment, sorted by appointment date, and limited to one row. The query joins the patients, appointments, and doctors tables based on their IDs, and then orders the results by the appointment date. The LIMIT clause is used to limit the number of rows returned to one, which means the query will only return the first row of the sorted result set. [S1] The join operators used in this query are interchangeable, and the CROSS JOIN operator is not used here. [S2] The ORDER BY clause is applied to the last simple SELECT in the compound query, which is the SELECT of appointments, and it sorts the rows based on the appt_date column. The rows are first sorted by appt_date, and then ties are broken by the next expression in the ORDER BY list, which is not present here. [S3] The LIMIT clause is applied to the last simple SELECT in the compound query, which is the SELECT of appointments, and it limits the number of rows returned to one. The LIMIT expression evaluates to an integer, which is 1, and the query returns the first row of the sorted result set.
Judge: The explanation does not cover every aspect of the query. It misses explaining the join conditions and the use of LIMIT correctly.

---

q100 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY patients.birth_year LIMIT 1;
```

Baseline: bad
This SQL query retrieves the name of a patient, the name of the doctor who saw them, the date of the appointment, and the fee for that appointment. The query starts by joining the patients table with the appointments table on the patient's ID, and then joins the appointments table with the doctors table on the doctor's ID. The result is then ordered by the patient's birth year, so the earliest birth year comes first. The query then limits the result to just one row, effectively returning the information for the patient with the earliest birth year.
Judge: The explanation does not cover the ORDER BY clause and LIMIT 1, which are part of the query.

RAG: good
This query returns the name of the patient, the name of the doctor, the appointment date, and the fee for the most recent appointment, sorted by the patient's birth year in ascending order, and limited to one row.
Judge: The explanation correctly describes the query's operations, including joins, filters, grouping, aggregate, ordering, and LIMIT. It uses clear language and covers all aspects of the query.

---

q101 · JOIN of three tables with ORDER BY and LIMIT

```
SELECT patients.name, doctors.name, appointments.appt_date, appointments.fee FROM patients JOIN appointments ON patients.id = appointments.patient_id JOIN doctors ON appointments.doctor_id = doctors.id ORDER BY patients.name LIMIT 5;
```

Baseline: good
This query retrieves the names of patients, doctors, appointment dates, and fees from the database. It joins three tables: patients, appointments, and doctors. The query starts by matching patients with their corresponding appointments based on the patient's ID, and then matches appointments with doctors based on the doctor's ID. The results are ordered alphabetically by patient name, and only the first 5 rows are returned. The query produces the names of patients, doctors, appointment dates, and fees, with no additional calculations or aggregations. The appointment dates are returned as strings, and the fees are returned as real numbers.
Judge: The explanation correctly describes the query's functionality, covers all aspects of the query (joins, filters, grouping, aggregate, ordering, and LIMIT), and uses clear language.

RAG: bad
This query returns the name of patients, doctors, appointment dates, and fees, sorted by patient name, and returns only the first 5 rows. The query joins the patients, appointments, and doctors tables based on their IDs, and uses a CROSS JOIN to combine the tables. The results are ordered by patient name in ascending order, and the first 5 rows are returned.
Judge: The explanation incorrectly states a CROSS JOIN was used, when in fact a JOIN was used. The query structure is correct, but the explanation is misleading.